

Doc no: P1072R0
Date: 2018-05-04
Reply to: Chris Kennelly (ckennelly@google.com), Mark Zeren
(mzeren@vmware.com)
Audience: LEWG/LWG/SG16

Default Initialization for basic_string

[Summary](#)

[Motivation](#)

[Proposal](#)

[Option A](#)

[Option B](#)

[Related Work](#)

Summary

Extend `basic_string` to allow access to default-initialized elements.

We propose similar changes to `vector` in [P1010R0](#).

Motivation

Performance sensitive code is impacted by the cost of manipulating strings:
When streaming data into a `basic_string`, a programmer is forced to choose between extra initialization (resize then copy directly in) or extra copies (copy into a temporary buffer, then append).

Consider writing a pattern several times into a string:

```
std::string GeneratePattern(const std::string& pattern,
size_t count) {
    std::string ret;
    ret.reserve(pattern.size() * count);
    for (size_t i = 0; i < count; i++) {
        ret.append(pattern); // BAD: Extra bookkeeping
    }
    return ret;
}
```

Alternatively, we could adjust the output string's size to its final size, avoiding the bookkeeping in `append` at the cost of extra initialization:

```
std::string GeneratePattern(const std::string& pattern,
    size_t count) {
    std::string ret;
    const auto step = pattern.size();
    ret.size(step * count); // BAD: Extra
    initialization
    for (size_t i = 0; i < count; i++) {
        // GOOD: No bookkeeping
        memcpy(ret.data() + i * step, pattern.data(),
    step);
    }
    return ret;
}
```

We propose adding an interface to `basic_string` to avoid this tradeoff:

```
std::string GeneratePattern(const std::string& pattern,
    size_t count) {
    std::string ret;
    const auto step = pattern.size();
    // GOOD: No initialization
    ret.resize_uninitialized(step * count);
    for (size_t i = 0; i < count; i++) {
        // GOOD: No bookkeeping
        memcpy(ret.data() + i * step, pattern.data(),
    step);
    }
    return ret;
}
```

Google has implemented `resize_uninitialized` (Option A) in its standard library. This is used in performance critical sections of code such as:

- [Abseil](#): [StrCat](#) and [StrAppend](#) can copy bytes directly into the final string, eliding size checks that `basic_string::append` would perform when adding each piece. These uses are similar to the examples illustrated in this section.
- [Protocol Buffers](#): Extraneous copies or initialization are avoided where the size is known before the data is available. On [serialization](#), the size has been calculated (needed for length prefixed data) before the contents are ready. On [deserialization](#), the size will have been parsed and bytes need to be copied into a string for user code to access.
- [Snappy](#): During [decompression](#), the final size of the output buffer is known before the contents are ready. During [compression](#), an upperbound on the final compressed size is known, allowing data to be efficiently added to the output buffer (eliding `append`'s checks) and the string to be shrunk to its final, correct size.

Proposal

Option A

Add to `basic.string.capacity` [24.3.2.4]:

`void resize_uninitialized(size_type n);`

- *Throws*: `length_error` if `n > max_size()`
- *Effects*: Alters the length of the string designated by `*this` as follows:
 - If `n <= size()`, calls `resize(n)`
 - If `n > size()`, the function replaces the string designated by `*this` with a string of length `n` whose first `size()` elements are a copy of the original string designated by `*this`, and whose remaining elements are default initialized.

The “null terminator” invariant of `basic_string` [24.3.2] is unchanged.

Option B

While there is implementation experience for Option A, Option B may be more friendly to processor memory prefetchers on modern architectures, particularly when memory is being initialized from `uninitialized_data()` in order for long strings.

Add to `string.accessors` [24.3.2.7.1]

`charT* uninitialized_data() noexcept;`

- Returns a pointer `p` such that `p == data() + size()` and `[p, p + capacity() - size())` is a valid range. All values except `*p` have indeterminate value. Modifying `*p` requires a subsequent call to `insert_from_capacity()` to restore the “null terminator.”
- Complexity: Constant time

Add:

`basic_string& insert_from_capacity(size_type n);`

- *Throws*: `length_error` if `n > capacity() - size()`
- *Effects*: Appends `n` elements that must have been previously initialized by the application via `uninitialized_data()`. `data() + size()` points to an object with value `charT()`.

Related Work

- [P0593](#): Bless implicit creation of trivial types

- [P1010](#): Proposal to expose default-initialized elements for vector